GitHub Copilotを使った
既存リポジトリへの有効活用方法

～レガシーコードを段階的に改善する実践アプローチ～

© 2026

本日のアジェンダ

1. 🎯 よくある課題：テストもリンターもないコード
2. 🧪 Step 1：テストの導入
3. 📏 Step 2：リンターの導入
4. 🔄 Step 3：CI/CDの構築
5. 📝 Step 4：Copilot指示書の作成
6. 💡 まとめ

© 2026

想定する状況：「よくあるレガシーリポジトリ」

```python
# app.py - FlaskベースのタスクAPIの例
from flask import Flask, request, jsonify
import sqlite3
import os   # ← 未使用のimport

app = Flask(__name__)

@app.route('/tasks', methods=['GET'])
def get_tasks():
    conn = sqlite3.connect('tasks.db')  # テストなし、CIなし
```

問題点： テストなし / リンターなし / CI/CDなし → 変更が不安

© 2026

Step 1：テストの導入 🧪

Copilotにテスト生成を依頼

やったこと：

- `tests/conftest.py` — テストフィクスチャ（一時DB生成）
- `tests/test_api.py` — 各エンドポイントの結合テスト（18テスト）
- `tests/test_scenarios.py` — 複数操作の連結テスト（4テスト）

ポイント：

- 既存コード（`app.py`）は **一切変更しない**
- `tmp_path` を使い本番DBに影響しない
- 日本語のテスト名で可読性向上

© 2026

Step 1：テスト結果

```
tests/test_api.py::TestCreateTask::test_タスクを正常に作成 PASSED
tests/test_api.py::TestDeleteTask::test_タスクを正常に削除 PASSED
tests/test_api.py::TestSearchTasks::test_タスクを検索 PASSED
tests/test_api.py::TestToggleTask::test_タスクの完了状態をトグル PASSED
tests/test_scenarios.py::TestTaskLifecycle::test_タスクの作成_更新_完了_削除フロー PASSED
...
========================= 22 passed in 0.21s =========================
```

✅ **22テスト 全てパス** — 既存コードの変更なしにテストカバレッジを確保

© 2026

Step 2：リンターの導入 📏

Copilotにruff設定を依頼 → `pyproject.toml` を自動生成

```toml
[tool.ruff]
target-version = "py310"
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "UP", "B", "SIM"]

[tool.ruff.lint.per-file-ignores]
"tests/**" = ["N802"]  # 日本語テスト名を許可
```

検出・修正： 未使用import削除 / importソート / フォーマット統一

© 2026

Step 3：CI/CDの構築 🔄

Copilotにワークフローを依頼

```yaml
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main, "step/**"]
jobs:
  lint:     # uv run ruff check + format check
  test:     # uv run pytest（lintパス後に実行）
```

ポイント： `astral-sh/setup-uv` で環境非依存にセットアップ

`uv sync --frozen` でロックファイルから再現可能な環境を構築

© 2026

Step 4：Copilot指示書の作成 📝

`.github/copilot-instructions.md`

```
# Copilot Instructions
## プロジェクト概要
Flaskベースのタスク API です。
## コーディング規約
- ruffの設定(pyproject.toml)に従うこと
- テストの関数名は日本語で記述してよい
## 開発時の注意事項
- 新規エンドポイント追加時はtest_api.pyにテスト追加
- コミット前にuv run ruff check .を実行
- 依存追加はuv addで行う
```

効果： Copilotがプロジェクト固有のルールを理解し、的確なコード生成が可能に

© 2026

作業のビフォー・アフター

項目	Before	After
テスト	❌ なし	✅ 22テスト（結合＋連結）
リンター	❌ なし	✅ ruff（lint + format）
CI/CD	❌ なし	✅ GitHub Actions
Copilot指示書	❌ なし	✅ copilot-instructions.md
コード品質管理	🙋 手動	🤖 自動化

© 2026

Copilot活用のポイント

1. 既存コードを壊さない

テスト追加時に既存コードは一切変更しなかった

2. 段階的に改善する

テスト → リンター → CI/CD → 指示書の順で段階的に導入

3. 指示書でCopilotを「育てる」

`copilot-instructions.md` でプロジェクト固有の文脈を共有

4. ブランチで変更を可視化

各改善ステップをブランチで分け、差分を明確にする

© 2026

まとめ

GitHub Copilotは「新規開発」だけでなく「既存改善」にも強力なツール

1. **テスト生成** — 既存APIの仕様を読み取り、包括的テストを生成
2. **リンター導入** — 設定ファイルの生成から自動修正まで
3. **CI/CD構築** — プロジェクトに最適なワークフローを提案
4. **指示書作成** — 自身への指示を自分で作れる

> 💡 **「コードを書く」だけでなく「コードベースを良くする」ためにCopilotを使おう**

© 2026

ブランチ構成（デモリポジトリ）

```
main
  └─ step/1-api            ← 素のFlask API（テスト・リンターなし）
       └─ step/2-tests     ← テスト追加
            └─ step/3-linter    ← リンター導入
                 └─ step/4-cicd    ← CI/CD追加
                      └─ step/5-copilot-instructions ← 指示書追加
                           └─ step/6-slides ← 発表資料
                                └─ step/7-pr-template ← PRテンプレート
                                     └─ step/8-branch-protection ← ブランチ保護
```

各ブランチの差分で「何が追加されたか」が一目瞭然

© 2026

Tips：CopilotにPR文を自動生成させる 💡

仕組み

1. `.github/pull_request_template.md` にPRテンプレートを用意
2. `copilot-instructions.md` に「PR作成ガイド」を記載
3. 開発完了後にCopilotへ一言依頼するだけ

依頼例

> 「このブランチの変更内容を元に、PRテンプレートに沿ったPR文を作成して」

Copilotが差分を読み取り、概要・変更内容・テスト項目を自動で埋めてくれる

© 2026

Tips：ブランチ保護をコードベースで管理する 🛡️

仕組み

1. `.github/branch-protection.json` に保護ルールをJSONで定義
2. PRがmainにマージされると **GitHub Actionsが自動適用**
3. ルール変更はJSONを編集 → PRを出すだけ

保護ルールの内容

PRレビュー必須 / ステータスチェック必須（Lint, Test） / Force Push禁止 / 管理者にも適用

ご清聴ありがとうございました 🙏

リポジトリ： このリポジトリをクローンして各ブランチを確認してください

質問はありますか？

© 2026